



Sviluppo Applicazioni Mobili
V. Gervasi – a.a. 2023/24



Sviluppo di Applicazioni Mobili

Vincenzo Gervasi
Dipartimento di Informatica

Email: gervasi@di.unipi.it

Sito web: <http://www.di.unipi.it/~gervasi>

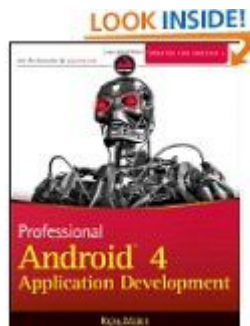
Ufficio: Stanza 367, Dipartimento di Informatica



Logistica del corso

- 6 CFU – 2° semestre
- INF-L complementare (sugg. 3° anno)
- Orario delle lezioni:
 - Giovedì 16:00-18:00 aula Fib L1 (?)
 - Venerdì 16:00-18:00 aula Fib C1 (?)
- Orario di ricevimento:
 - Martedì 11:00, stanza 367 Dip. Informatica
 - Su appuntamento per email, in caso di impossibilità di usufruire del ricevimento ordinario

Testi raccomandati

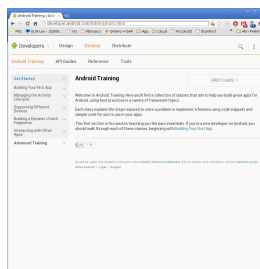


- **Reto Meier**, *Professional Android 4|2|ε Application Development*, Wrox Publishing, 2012|2010|2008

obsoleti



- **E. Di Saverio, S. Sanna**, *Android. Programmazione avanzata*, Edizioni FAG, 2012



- Tutorial online:
<http://developer.android.com/training>



Programma di massima del corso



- 1 Introduzione al corso, logistica, modalità d'esame. Breve storia di Android, stato del mercato, l'ambiente di sviluppo, dettagli sull'installazione.
- 2 Architettura di un sistema Android: kernel, sandbox, framework. Linux, Dalvik, ART, e la sicurezza. Struttura di un'applicazione Android (in sviluppo, in deployment, in esecuzione).
- 3 I sistema delle risorse: architettura generale, risorse alternative, esempi. AndroidManifest.xml. Hello World!
- 4 Componenti di un'applicazione: activity, service, broadcast receiver, content provider. Comunicazione fra componenti attraverso Intent. Ancora su AndroidManifest.xml. Dettagli su Activity. Un esempio completo: BMI Calc
- 5 Il ciclo di vita di un'Activity. Salvataggio e ripristino dello stato.
- 6 Definire la UI di un'Activity. Layout & View. Interazione (con richiami sui Listener). Il sistema dei menu: options menu, menu contestuali, pop-up, contextual action bar, popolamento in base a Intent.: Scrivere proprie View; negoziazione del layout, repaint. La pipeline di rendering 2D e le classi Canvas e Paint. Un esempio completo. Scrivere propri layout manager. Esempio: EqLayout. Temi e stili. ListView statiche e dinamiche. Data adapter e binding. Altri usi di adapter. RecyclerView: pattern e usi tipici. La classe WebView: uso, configurazione, specializzazione; binding fra Java e Javascript. La classe Drawable e le sue sottoclassi (con dettagli su 9 patch). Notifiche all'utente: Toast, Snackbar e Notification. Dialog, Fragment, DialogFragment.
- 7 Tematiche di storage. Storage temporaneo: salvataggio temporaneo dello stato, Bundle e Parcelable. Storage permanente: preferenze, PreferenceScreen e PreferenceActivity; accesso al file system. Accesso a database. Il DBMS SQLite e la classe SQLiteDatabase. Eseguire statement SQL. Query e la classe Cursor. Incapsulare l'accesso al database: la classe SQLiteOpenHelper e i pattern di progettazione relativi. Condivisione di dati su Android. ContentProvider e ContentResolver. Accesso a ContentProvider di sistema; il meccanismo delle URI. Scrivere ed esportare propri ContentProvider. La gestione dei permessi.
- 8 Multithreading. Ripasso / anticipazione sul multithreading in Java: Thread e Runnable, controllo di thread, meccanismi di notifica (wait/notify) e serializzazione (synchronized). La classe AsyncTask. Altri approcci all'esecuzione asincrona: post() e runOnUiThread(). Struttura del dispatcher di Android: Handler, Looper, messaggi e code. Casi tipici di programmazione in background: uso di AsyncTask per il caricamento di immagini da web; i Loader e l'uso di cursori asincroni; uso del servizio DownloadManager e del ContentProvider associato; AsyncPlayer. Esempi.
- 9 Segnalazioni broadcast; i Broadcast Receiver. Registrazione statica (in XML) e dinamica (a run-time) di un Receiver. Invio di messaggi broadcast. Le varianti con permessi, serializzazione, terminazione, raccolta risultati, persistenza. Gli allarmi. Il servizio AlarmManager e le varie tipologie di allarmi (con wake-up, ripetuti, imprecisi).
- 10 Esecuzione in background: i Services. Ciclo di vita di un Service. Servizi started (con approfondimenti su IntentService) e bound. Esempio: il Random Music Player.
- 11 Accesso ai sensori. Sensor discovery; i diversi tipi di sensori; ottenere le letture tramite callback. Pooling dei SensorEvent e cautele relative. Esempio: Sensors.java.
- 12 Tecnologie di rete. Networking TCP/IP in Java. Il ConnectivityManager. Bluetooth: introduzione, discovery, pairing, comunicazioni. Wi-Fi Direct: design del framework, discovery, connessione, trasferimento. Cenni su NFC.
- 13 Servizi di localizzazione, geocoding, mappe.
- 14 Architettura multimediale: audio, foto, video.
- 15 Programmazione nativa. Google APIs: esempi, uso via REST, uso via Google Play Services.

Soggetto a modifiche
(aggiornamenti
durante il semestre)

Modalità d'esame

- Sviluppo di una app
 - Tema proposto dallo studente
 - Dettagli concordati in anticipo con il docente
- Esame orale consistente in
 - Presentazione della app
 - Ispezione del codice
 - Domande “di teoria” su aspetti non coperti nel progetto
- Non sono previsti “compitini” o altre attività di verifica intermedia



ANDROID

Lezione 1



Programmazione Android



- Breve storia di Android
- Ambienti di sviluppo
 - Eclipse + ADT
 - Android Studio
 - Dettagli sull'installazione
- Architettura di un sistema Android
 - Kernel
 - La macchina virtuale
 - Librerie e Framework

Breve storia di Android

Breve storia di Android



- Rewind al 2007
 - Palm († 2006)
 - Windows CE (1996-2011)
 - Blackberry (1999-2015)



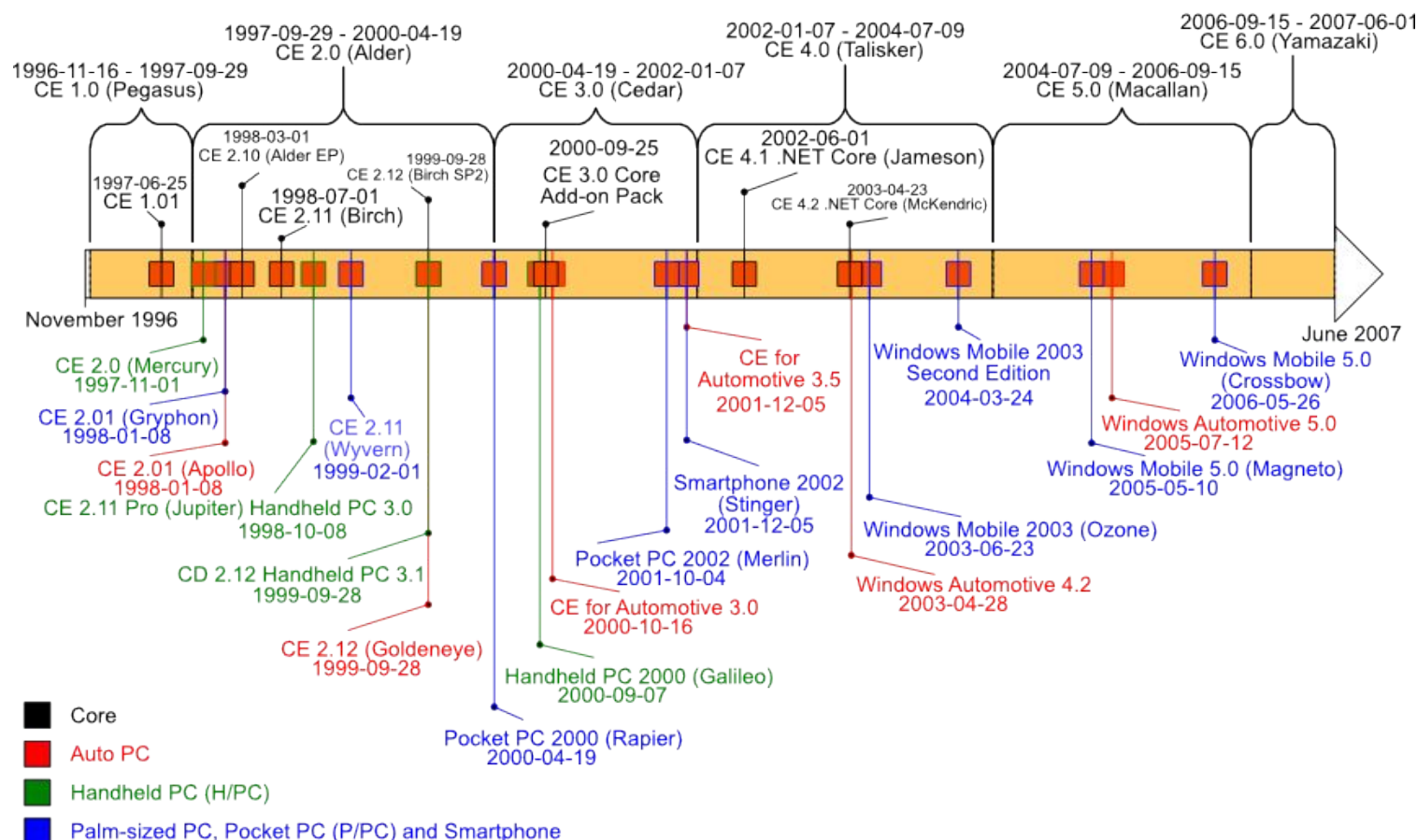
Breve storia di Android



- Dilemma per i produttori
- Tutti sistemi **fortemente proprietari**
- JavaME?
 - Portabile
 - Molto limitato

Windows CE Timeline

Source: "A Brief History of Windows CE" (<http://www.hpcfactor.com/support/windowsce/>), HPC: Factor, retrieved May 21, 2007



Breve storia di Android



- Novembre 2007: un gruppo di produttori di telefoni forma la **Open Handset Alliance**

Industry Leaders Announce Open Platform for Mobile Devices

November 5, 2007

Group Pledges to Unleash Innovation for Mobile Users Worldwide

MOUNTAIN VIEW, Calif.; BONN, Germany; TAOYUAN, Taiwan; SAN DIEGO, Calif.; SCHAUMBERG, Ill., November 5, 2007 — A broad alliance of leading technology and wireless companies today joined forces to announce the development of Android, the first truly open and comprehensive platform for mobile devices. Google Inc., T-Mobile, HTC, Qualcomm, Motorola and others have collaborated on the development of Android through the Open Handset Alliance, a multinational alliance of technology and mobile industry leaders.

Open Handset Alliance Founding Members

Aplix (www.aplixcorp.com), Ascender Corporation (www.ascendercorp.com), Audience (www.audience.com), **Broadcom** (www.broadcom.com), **China Mobile** (www.chinamobile.com), **eBay** (www.ebay.com), Esmertec (www.esmertec.com), **Google** (www.google.com), **HTC** (www.htc.com), **Intel** (www.intel.com), KDDI (www.kddi.com), Living Image (www.livingimage.jp), **LG** (www.lge.com), Marvell (www.marvell.com), **Motorola** (www.motorola.com), NMS Communications (www.nmscommunications.com), Noser (www.noser.com), **NTT DoCoMo, Inc.** (www.nttdocomo.com), Nuance (www.nuance.com), **Nvidia** (www.nvidia.com), PacketVideo (www.packetvideo.com), **Qualcomm** (www.qualcomm.com), **Samsung** (www.samsung.com), SiRF (www.sirf.com), SkyPop (www.skypop.com), SONiVOX (www.sonivoxrocks.com), **Sprint Nextel** (www.sprint.com), Synaptics (www.synaptics.com), TAT - The Astonishing Tribe (www.tat.se), **Telecom Italia** (www.telecomitalia.com), **Telefónica** (www.telefonica.es), **Texas Instruments** (www.ti.com), **T-Mobile** (www.t-mobile.com), Wind River (www.windriver.com)

Breve storia di Android



Open Handset Alliance Releases Android SDK

November 12, 2007

The Open Handset Alliance, a group of mobile and technology leaders, today announced the availability of the Android Software Development Kit (SDK). Available now as an early look, the Android SDK will enable developers to create innovative and compelling applications for the platform. The early look will also provide developers with the opportunity to participate in the evolution of the Android platform by providing feedback throughout the development process.

The Android platform was built from the ground up to enable developers to create new and innovative mobile applications that take full advantage of all the capabilities of a handset connected to the internet. It is a complete mobile platform built on the Linux 2.6 kernel that exposes a robust operating system, a comprehensive set of libraries, a rich multimedia user interface, and a complete set of phone applications. Android's innovative application model makes it easy for developers to extend, replace, and reuse existing software components to create rich and integrated mobile services for consumers.

The Android platform also includes the Dalvik virtual machine to maximize application performance, portability, and security. The entire platform will be made available under the very liberal, developer-friendly Apache v2 open-source license in 2008.

Android Software Development Kit

The SDK contains a rich set of tools for developers to build applications for the Android platform. Included are advanced development and debugging tools, a rich set of libraries, a true device emulator, in-depth documentation, sample projects, tutorials, FAQs, and more. For developers looking for a seamless development experience, an Eclipse plugin is included to integrate these tools with the Eclipse integrated development environment. The site hosting the kit will also feature a blog and discussion groups, to make it easier for everyone contributing to the platform to interact and share knowledge.

Requirements

To begin building applications for Android, developers will need to download the Android SDK to an x86-based computer running Windows XP or Vista; Mac OS 10.4.8 or later; or Linux Ubuntu Dapper Drake or later (other modern distributions of Linux will also likely work but are not directly supported).

Developers will also need Eclipse 3.2 or later, with Java Development Tools and the Android SDK's plugin, or Java and Javac 1.5 or 1.6; Apache Ant; an integrated development environment; and Python 2.2 or later.

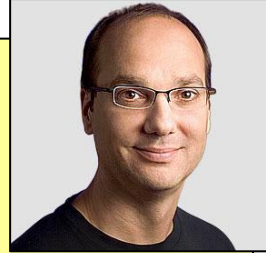
- 7 giorni dopo, viene rilasciato il primo SDK
 - Licenza Apache
- Basato su
 - Linux 2.6
- Sviluppo su
 - Eclipse
 - Java
 - Python (!)

Breve storia di Android



- In realtà, il software era stato già sviluppato
- Da Android Inc., la classica startup californiana
 - Nata nel 2003 a Palo Alto
 - Acquistata da Google nel 2005
 - Brevetti registrati nel 2007
- Quasi tutti i giochi erano già fatti a fine 2007
 - Architettura complessiva
 - Ambiente di sviluppo
 - Licensing
 - Partner (telefoni e carrier)

Breve storia di Android



Fondata da **Andy Rubin**

- In realtà, il software era s
- Da Android Inc., la class
 - Nata nel 2003 a Palo A
 - Acquistata da Google ne
 - Brevetti registrati nel 2007
- Quasi tutti i giochi erano
 - Architettura complessiva
 - Ambiente di sviluppo
 - Licensing
 - Partner (telefoni e carrier)

- 1986-1989 Carl Zeiss AG, robotics engineer
- 1989-1992 **Apple Inc.**, manufacturing engineer
- 1992-1995 General Magic, engineer. An Apple spin-off where he participated in developing Magic Cap, an operating system and interface for **hand-held mobile devices**.
- 1995-1999 **MSN TV**, engineer. When Magic Cap failed, Rubin joined Artemis Research, founded by Steve Perlman, which became WebTV and was eventually acquired by **Microsoft**.
- 1999-2003 Danger Inc., co-founder. Founded with Matt Hershenson and Joe Britt. Firm is most notable for the Danger Hiptop, often branded as the **T-Mobile Sidekick**, which is a phone with PDA-like abilities. Firm was later acquired by **Microsoft** in February 2008.
- 2003-2005 **Android Inc.**, co-founder.
- 2005-2014 **Google**. Senior Vice President in charge of **Android** for most of his tenure. Since December 2013, managing the robotics division of Google (which includes companies bought by Google, such as Boston Dynamics).
- 2014 Left Google to start an “incubator for hardware startups”
- 2015 Founder of Playground Global, an incubator “to help make advances in Artificial Intelligence (AI)” – \$300M capital
- 2016 Joined Essential, Inc. - maker of smartphones
- 2018 It becomes known that in 2014 he left Google to avoid a sexual harassment scandal
- 2020 Essential folds up
- 2021 Nothing, Inc. buys Essential’s brand

Breve storia di Android



- Dal 2007, sono state rilasciate numerose versioni
- Numero di versione e *codename*
 - Nomi di dolci
 - In ordine alfabetico!





Sviluppo Applicazioni Mobili
V. Gervasi – a.a. 2023/24

Versione corrente



La versione corrente è **Android 14**, rilasciata il 4 Ottobre 2023
Tipicamente viene rilasciata una nuova versione di Android ogni anno:

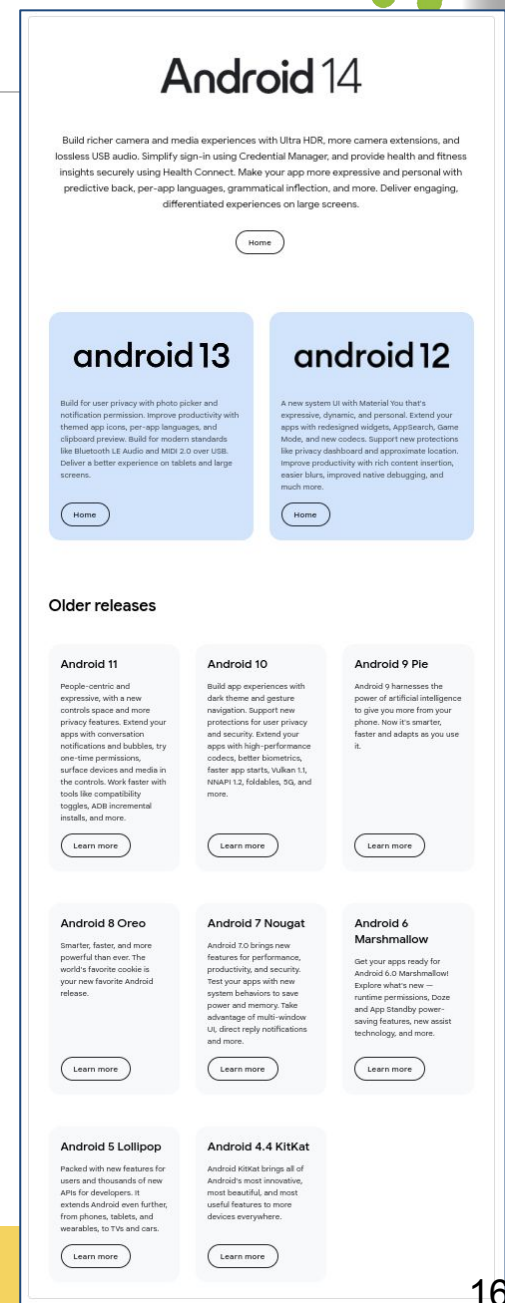
- preview in primavera
- finale in autunno

La prossima versione **Android 15** dovrebbe uscire:

- Developer Preview: Febbraio o Marzo (probabilmente a giorni)
- Beta: Aprile o Maggio
- Finale: fra Agosto e Ottobre

Da Android 10, è cessata la denominazione “ufficiale” con dolciumi, e si usa il numero (o la lettera: Android 10 è anche noto come “Android Q”)

Ai fini dell’esame: scegliete voi se scrivere app per l’ultimissima versione, o se mantenervi più cauti e “puntare” alle versioni precedenti



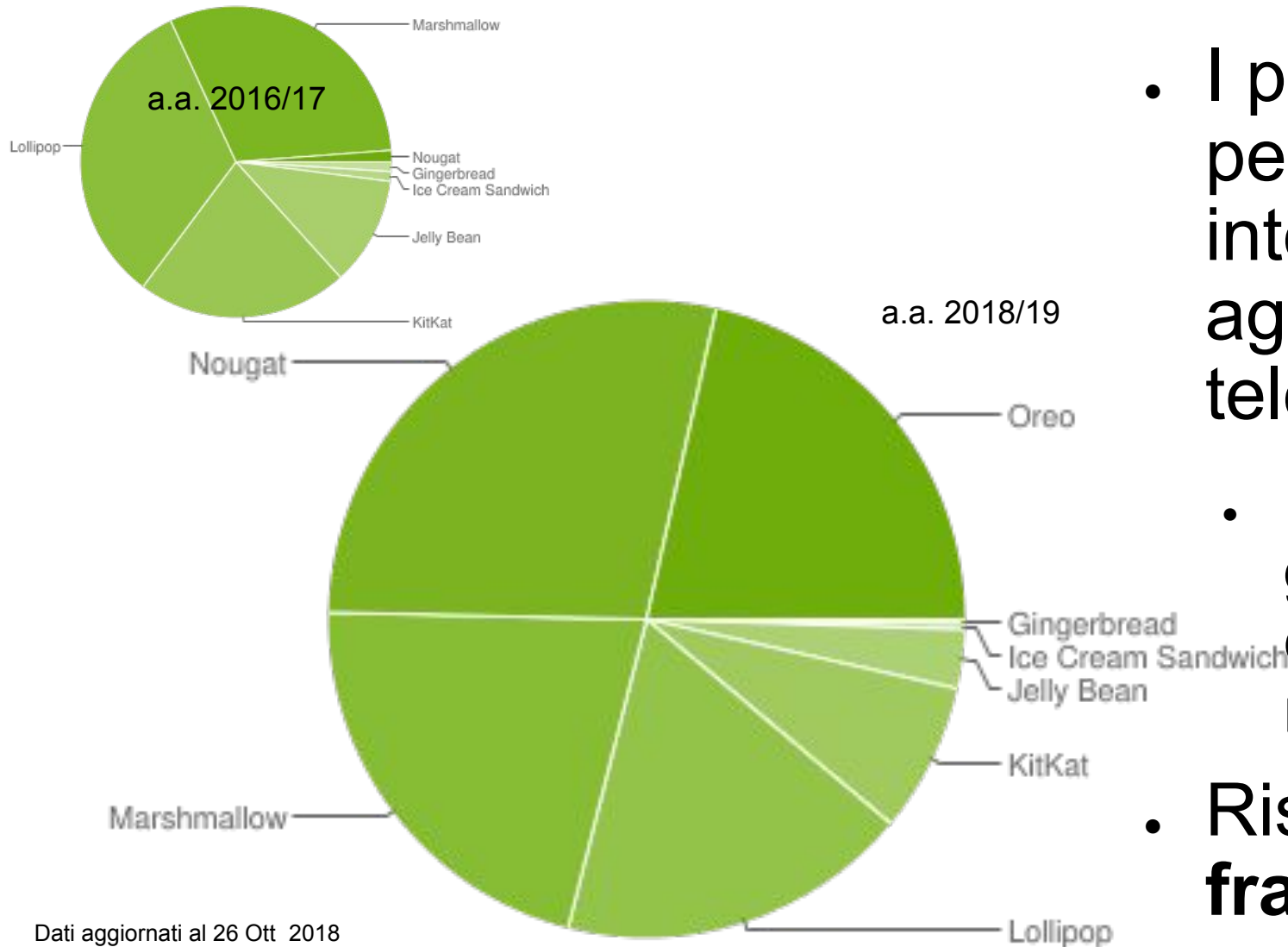
Breve storia di Android



- Da qui in avanti ci occuperemo quasi esclusivamente di **software**
- ... ma non bisogna dimenticare il ruolo dell'**hardware!**
 - Potenza di calcolo
 - Efficienza della batteria
 - Sensori
 - Schermi



Breve storia di Android



Dati aggiornati al 26 Ott 2018

- I produttori hanno però poco interesse ad aggiornare i telefonini “vecchi”
- Meglio spingere gli utenti a comprarne di nuovi!
- Risultato: **frammentazione**

Breve storia di Android



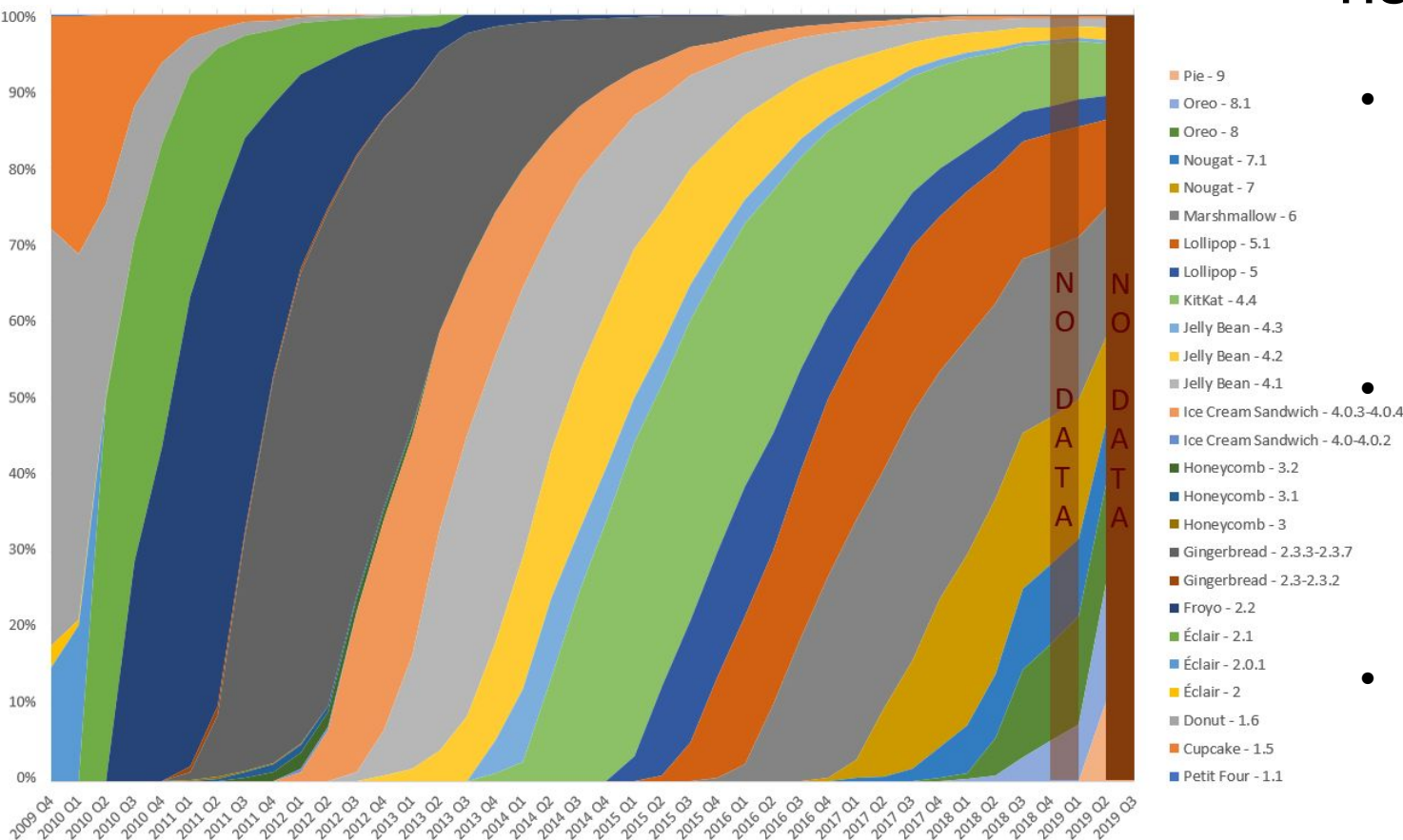
- Ogni versione successiva è (quasi) sempre pienamente compatibile con le precedenti
- Cambiamenti nelle API sono identificate da un **API Level**
- Le applicazioni possono dichiarare
 - Un API Level **minimo** di cui necessitano per funzionare
 - Un API Level **target** per cui sono state scritte
 - Un API Level **massimo** oltre il quale non funzionano più
 - *Pessima idea, sconsigliato, obsoleto, ignorato* da Android 2.0.1
- Il **Play Store** e il **Package Manager** del S.O. verificano il rispetto dei vincoli

Breve storia di Android



- Diverso panorama rispetto a iOS

- I device iOS vengono (quasi) sempre aggiornati alla versione più recente
- Android tende a diffondere aggiornamenti più lentamente
- L'Android più recente è cool, ma è sempre una nicchia!



Da Wikipedia; dati al Q3 2019

Breve storia di Android



- Google fa un *tentativo* di supportare più o meno all'infinito vecchie versioni del S.O.
 - Con le **librerie di compatibilità**
 - Codice che le applicazioni possono includere nel loro “eseguibile”
 - Simula le funzioni delle versioni più recenti su versioni antiche
 - Con i **Google Play Services**
 - Funzioni incorporate in una libreria aggiornabile da Market
- Grosso ostacolo: **customizzazione** (skinning)



Ambiente di sviluppo

Ambiente di sviluppo



- Possiamo considerare due livelli di sviluppo
 - Programmazione “nativa”
 - Si programma in C in ambiente Linux
 - GCC, librerie standard (libc, libm) e relativi header, librerie custom (liblogm, libjnigraphics) e relativi header
 - Tools per impacchettare codice nativo in formato **.apk**
 - Programmazione “standard”
 - Si programma in Java in ambiente Android (**non** Java!) oppure in Kotlin (che compila verso bytecode JVM)
 - Javac, parte delle librerie J2SE, molte librerie custom
 - Tools per trasformare il bytecode Java in DEX e impacchettare le classi in formato **.apk**



Ambienti di sviluppo integrati



- Ci sono sostanzialmente tre “IDE” in uso per Android
- CLI (toolchain)
 - Si sviluppa su command line, eseguendo in una shell ogni comando (compilatore, linker, ecc.)
- ~~Eclipse~~
 - Utile se vi serve Android + altri plugin, ma sempre più raro
 - Si usa Eclipse con dei plug-in ad-hoc per Android
- **Android Studio**
 - Default per Android “liscio”
 - Versione customizzata di IntelliJ IDEA



Sviluppo Applicazioni Mobile
V. Gervasi – a.a. 2023/24

Componenti dell'ambiente di sviluppo tipico - Eclipse



- **Java Development Kit (JDK)**
 - Il più testato è quello Sun (Oracle), in teoria potrebbero funzionare anche altre implementazioni – ma solo in teoria
- **Eclipse**
 - Più testati i package *Eclipse IDE for Java Developers* e *Eclipse Classic*
- **Android Development Tools (ADT)**
 - Plug-in per Eclipse per aggiungere il supporto ad Android
- **Android starter Software Development Kit (SDK)**
 - I tool specifici “core” per la programmazione in Android
- **Android Platform** e altri componenti simili (extra tools)
 - Contengono le immagini eseguibili delle varie versioni di Android, nonché altre librerie contenenti package di utilità (es., per accedere al Market)

Installazione - Eclipse



- È possibile installare separatamente JDK, Eclipse, ADT, ecc. e poi collegarli fra di loro
 - Utile solo se avete una installazione di Eclipse già molto configurata pre-esistente
- Ma è molto più comodo installare l'**ADT Bundle**
 - Eclipse + ADT plugin
 - Android SDK Tools
 - Android Platform-tools
 - A version of the Android platform
 - A version of the Android system image for the emulator

Installazione – Eclipse



Android Studio FEATURES USER GUIDE Search

ADT Plugin (DEPRECATED)

The Eclipse ADT plugin is no longer supported per our [announcement](#). Android Studio is now the official IDE for Android, so you should migrate your projects to Android Studio as soon as possible. For more information on transitioning to Android Studio, see [Migrate to Android Studio from Eclipse](#).

Android Developer Tools (ADT) is a plugin for Eclipse that provides GUI-based access to many of the command-line Android SDK tools. As with ADT, support for the [Ant](#) tool for building from the command line has ended. [Gradle](#) is now the supported method of building apps.

Revisions

The sections below provide notes about successive releases of the ADT Plugin, as denoted by revision number. For a summary of all known issues in ADT, see <http://tools.android.com/knownissues>.

^ [ADT 23.0.7 \(August 2015\)](#)

Dependencies:

- Java 7 or higher is required if you are targeting Android 5.0 and higher.
- Java 1.6 or higher is required if you are targeting other releases.
- Eclipse Indigo (Version 3.7.2) or higher is required.
- This version of ADT is designed for use with [SDK Tools r24.1.2](#). If you haven't already installed SDK Tools r24.1.2 into your SDK, use the Android SDK Manager to do so.

General Notes:

- Fixed issues with the rendering library for the visual layout editor.

▽ [ADT 23.0.6 \(March 2015\)](#)

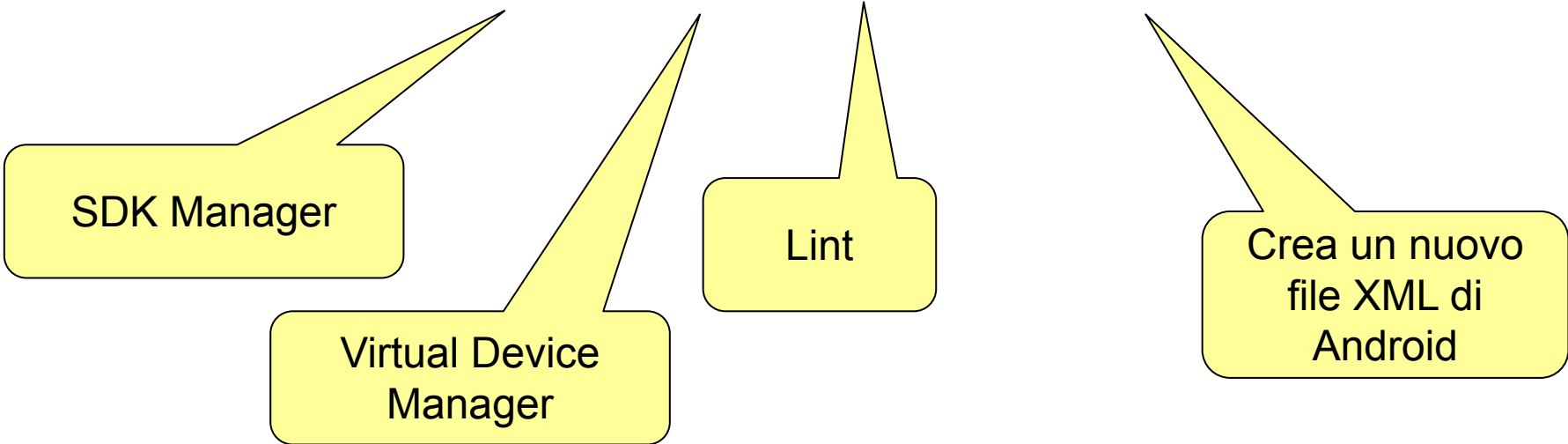
<https://developer.android.com/studio/tools/sdk/eclipse-adt.html>

Versione Linux 64 bit: 355Mb
Altri S.O. variano leggermente

Per chi non avesse
capito da che parte va il
mondo...

ADT su Eclipse

- Una volta installato l'ADT Bundle, si può aprire Eclipse
- Sulla toolbar troviamo quattro nuovi pulsanti:



SDK Manager

Virtual Device
Manager

Lint

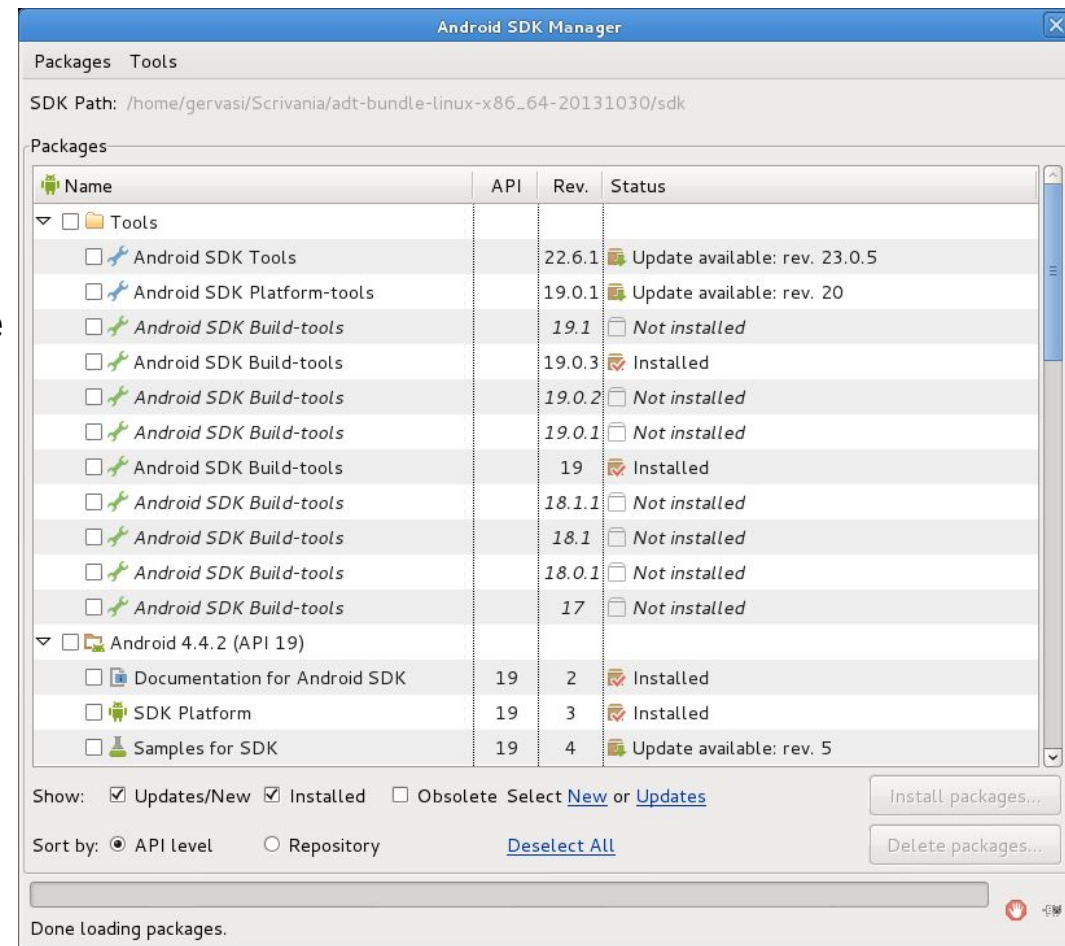
Crea un nuovo
file XML di
Android

ADT su Eclipse

• L'SDK Manager è un gestore di pacchetti interno

• Tipi di pacchetti:

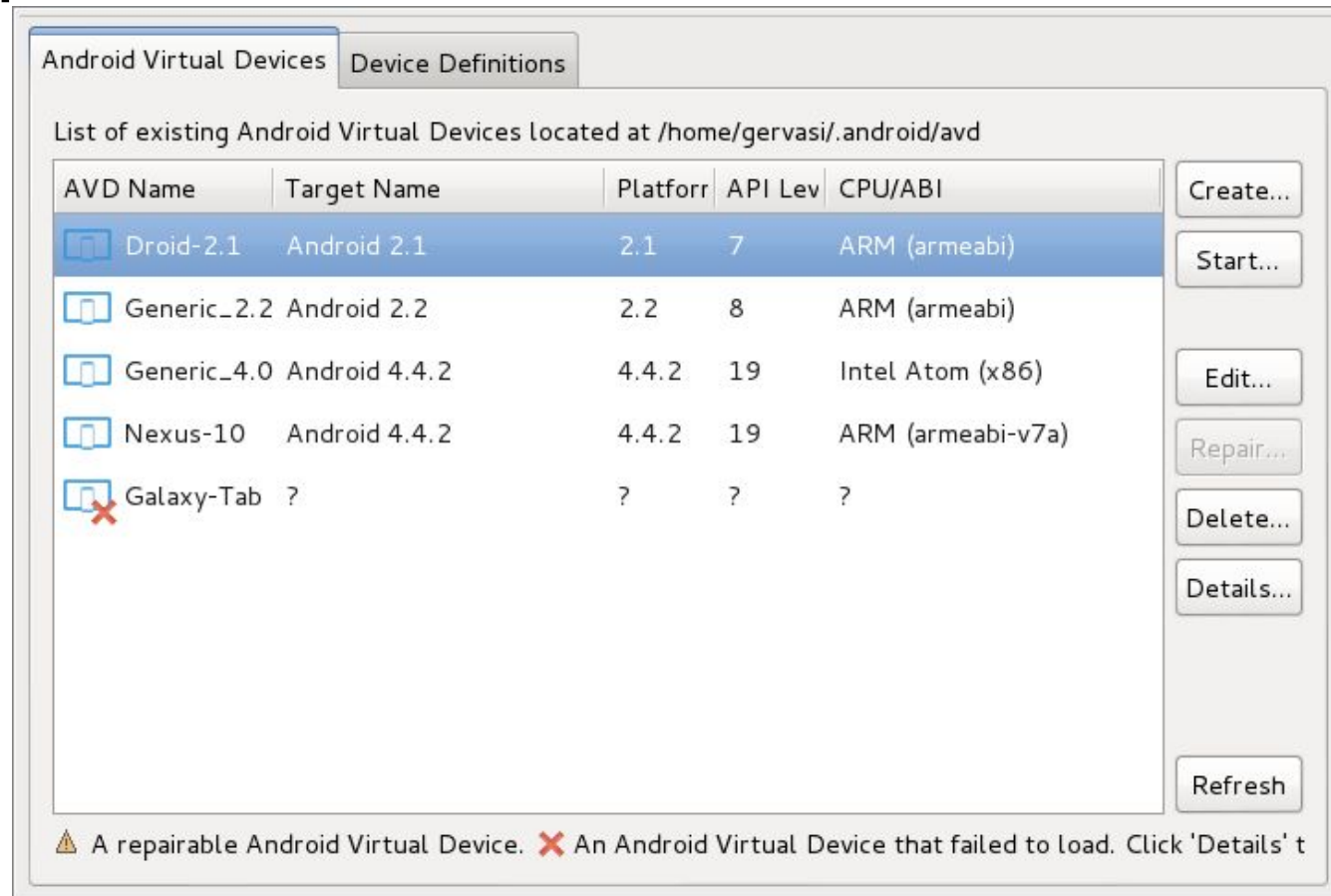
- Tool
 - Librerie
 - Immagini virtuali per l'emulatore
 - Esempi
 - Documentazione
- Altro
 - Librerie di terze parti
 - Librerie “esterne” di Google
 - Compatibilità
 - Google Play Services
 - AdSense
 - Ecc.



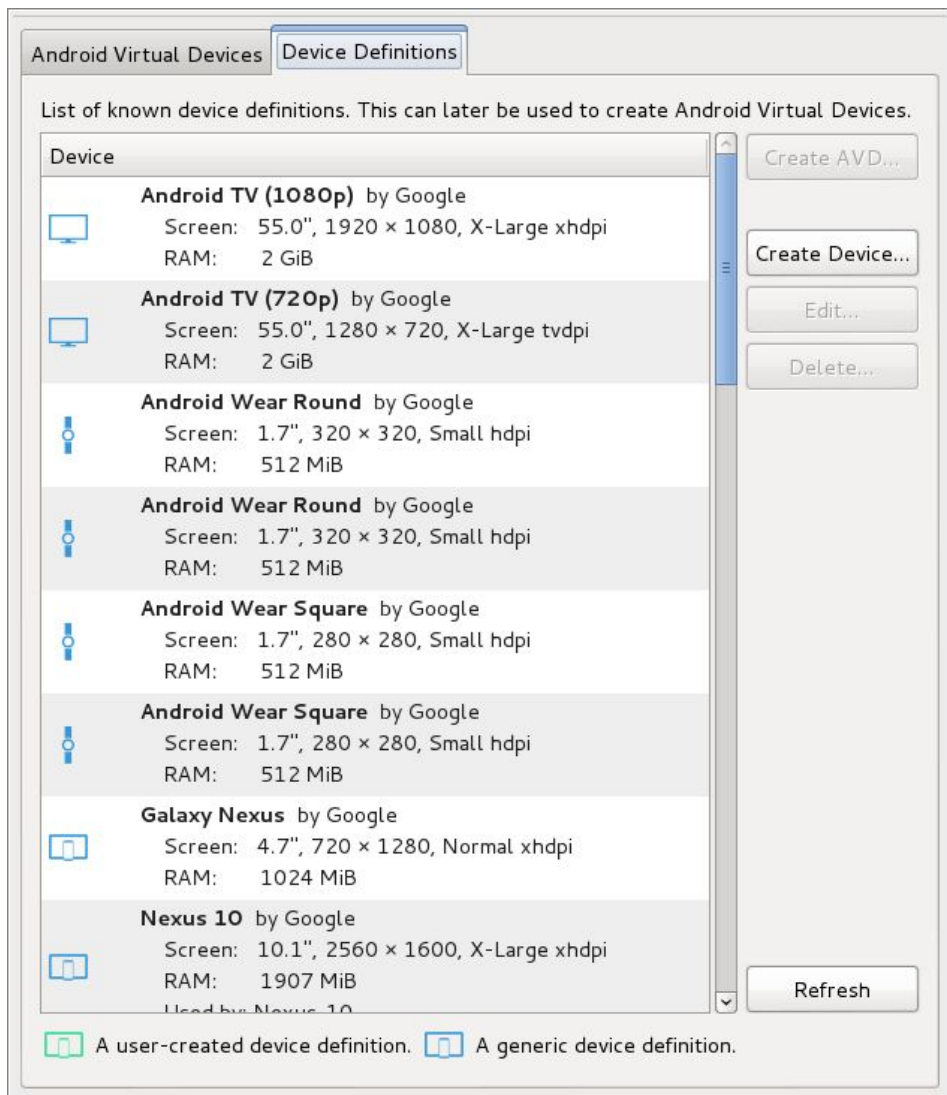
ADT su Eclipse

- L'**AVD Manager** gestisce le immagini dei vari dispositivi virtuali

- **Immagine** = file contenente una copia della memoria
- **Dispositivo virtuale** = configurazione per l'emulatore Android



ADT su Eclipse



- Vengono fornite **definizioni di dispositivo** per i casi più comuni
 - Android TV
 - Android Wear
 - Tutti i modelli NEXUS
 - Altri casi tipici
- Servono da base per creare i dispositivi virtuali (ciascuno con la sua immagine)

ADT su Eclipse



- Il **lint** è un venerabile tool per la ricerca **statica** di problemi nel codice
 - Vi segnala i casi più spesso problematici
 - Nel codice, nei file XML, nella struttura delle directory...
 - Non sempre sono **errori!**
- Il wizard “**Crea nuovo file XML**” vi offre dei template per i casi comuni
 - File di layout, menu, animazioni, preferenze, ecc.
 - Comodo, non indispensabile

Installazione – Android Studio



- In questo caso, esiste solo la versione “bundle”
- Include l'IDE vero e proprio, l'SDK, alcune immagini di default
- Funzionalmente, offre tutto quello che offre Eclipse
 - ambiente più moderno
 - un sistema di build più avanzato (e complicato)
 - tool per l'editing grafico di IU quasi utile
- È in sviluppo attivo (aggiornamenti frequenti)

Installazione – Android Studio



- Download da <https://developer.android.com/studio/index.html>
 - Altri 1.2Gb di .zip (o .dmg), consigliato 4Gb disco + 9Gb di RAM
- Offre i soliti tool
 - SDK Manager
 - AVD Manager
- Diversi l'editor, i wizard di refactoring, il sistema di build
 - Eclipse usa **Ant**, Android Studio **Gradle**



Sviluppo Applicazioni Mobili
V. Gervasi – a.a. 2023/24

Installazione – Android Studio



Tanto per non annoiarsi, le versioni di Android Studio hanno nomi in codici di animali

La versione corrente è la “**Android Studio Hedgehog (porcospino) 2023.1.1 Patch 2**”

Una volta installata, dovrete installare a cascata altri plugin:

- Android Emulator
- Android SDK Platform-Tools
- Android Gradle Plugin
- Google APIs ARM 64 v8a System Image (o la corrispondente versione amd64)
- ... ecc. a piacere

In genere circa 1.5-2.0Gb aggiuntivi

Android Studio e Gradle



- **Gradle** è un sistema di build avanzato, molto configurabile, adatto allo sviluppo distribuito
- Rispetto ad Ant, offre
 - **Varianti** multiple del prodotto finale
 - Per esempio: per diverse architetture
 - Dipendenze **remote** (tramite Maven)
 - Un artefatto può dipendere da una libreria di terze parti di cui si ha l'URL; il sistema controlla, scarica e aggiorna se necessario
 - **Fill-in del manifest**
 - “aggiusta” i file XML del manifest secondo il particolare build

Android Studio e Gradle



Scotto da pagare: una **maggior complessità** nella configurazione del build system.

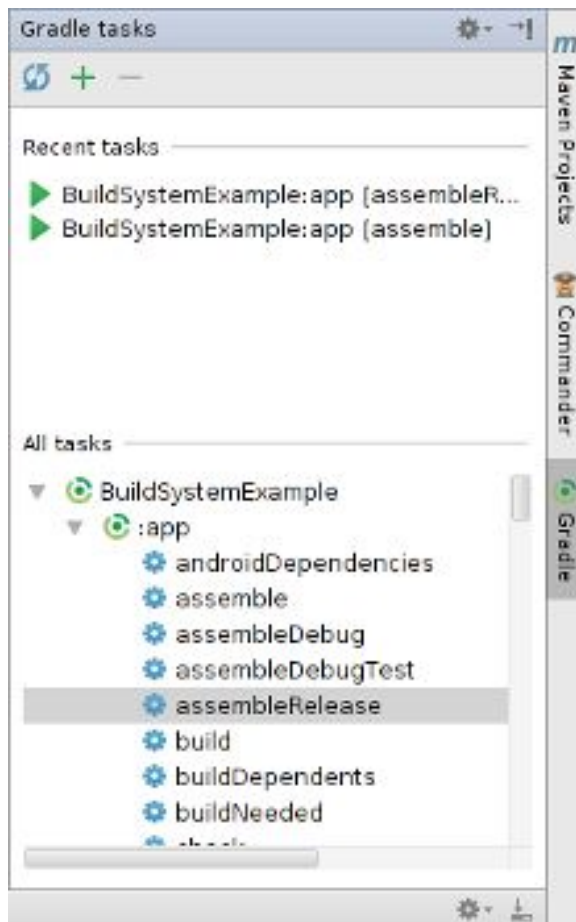
(Per fortuna molti default sono ragionevoli...)

```
apply plugin: 'android'

android {
    compileSdkVersion 19
    buildToolsVersion "19.0.0"

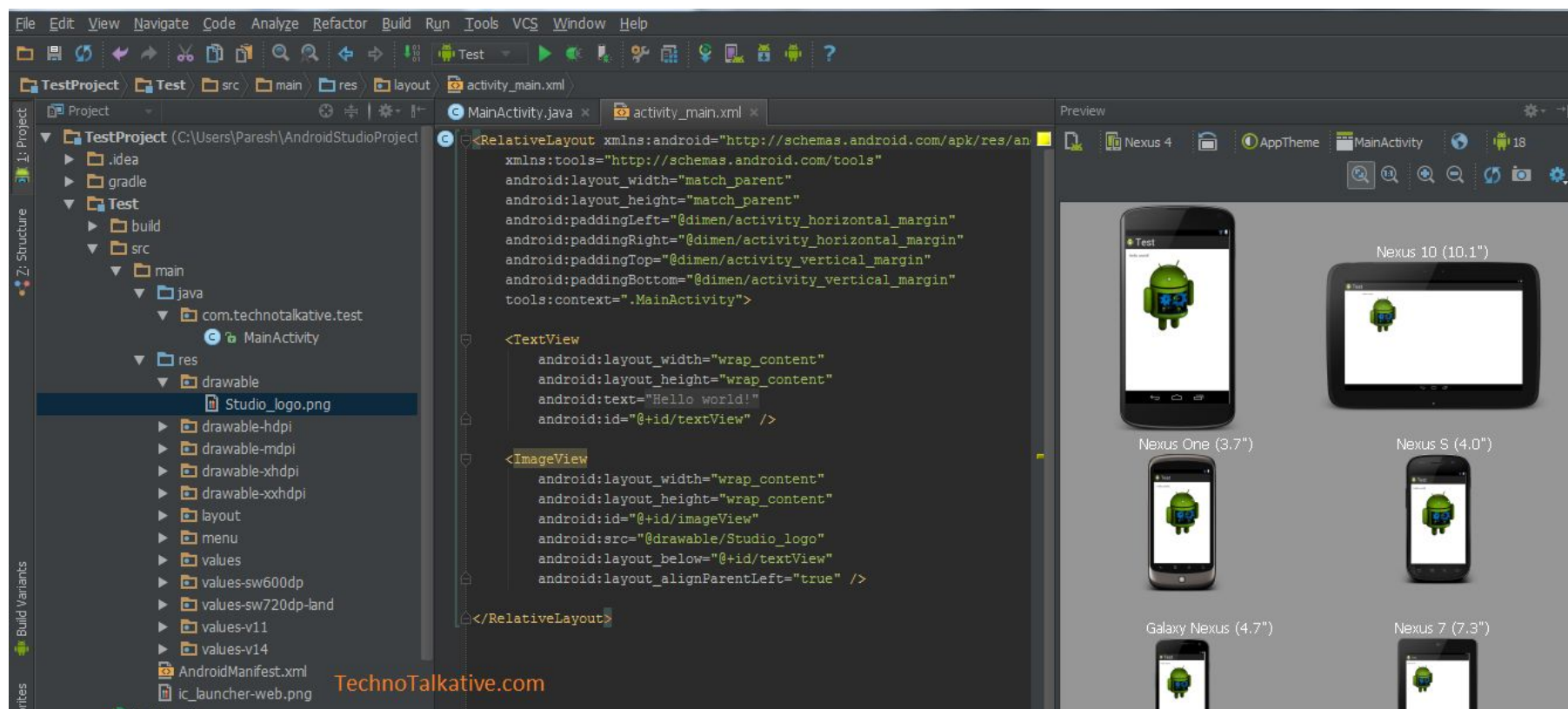
    defaultConfig {
        minSdkVersion 8
        targetSdkVersion 19
        versionCode 1
        versionName "1.0"
    }
    buildTypes {
        release {
            runProguard true
            proguardFiles getDefaultProguardFile('proguard-android.txt'), \
            'proguard-rules.txt'
        }
    }
}

dependencies {
    compile project(":lib")
    compile 'com.android.support:appcompat-v7:19.0.1'
    compile fileTree(dir: 'libs', include: ['*.jar'])
}
```



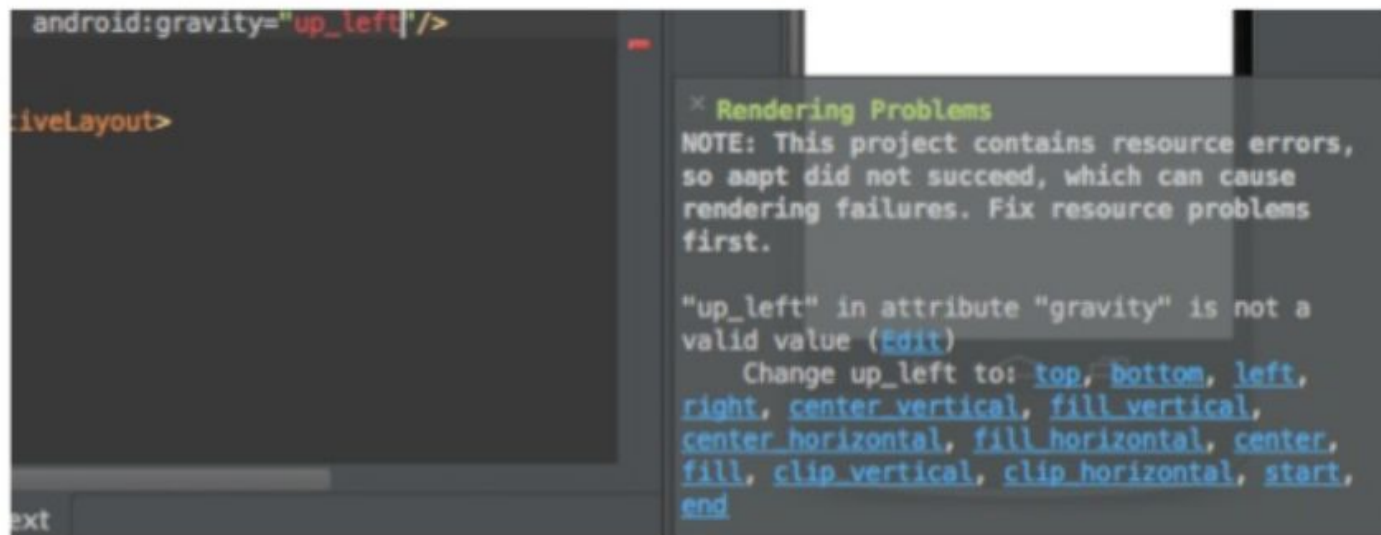
Android Studio

- Android Studio ha importanti vantaggi:
 - Design della UI in parallelo su più form factor

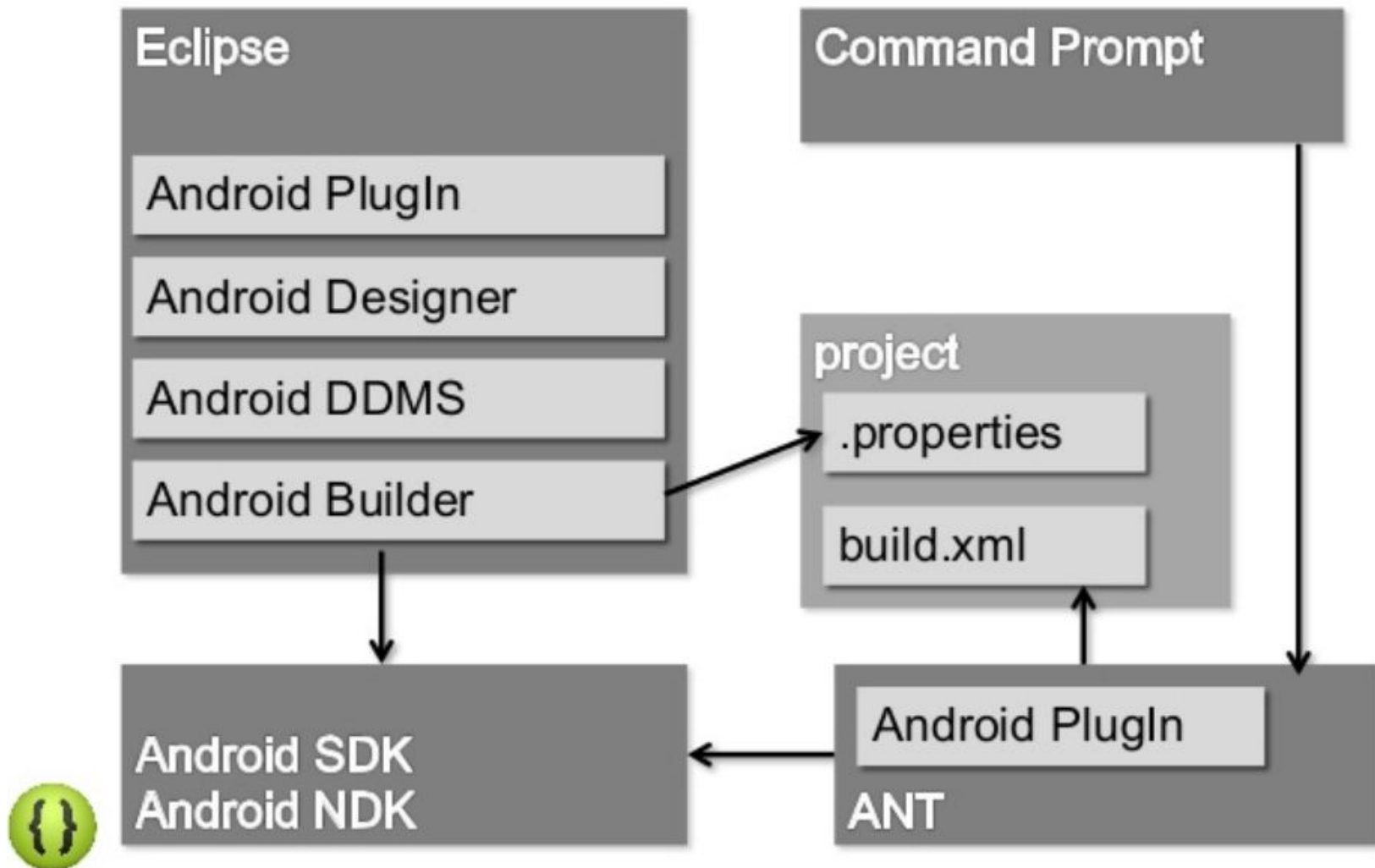


Android Studio

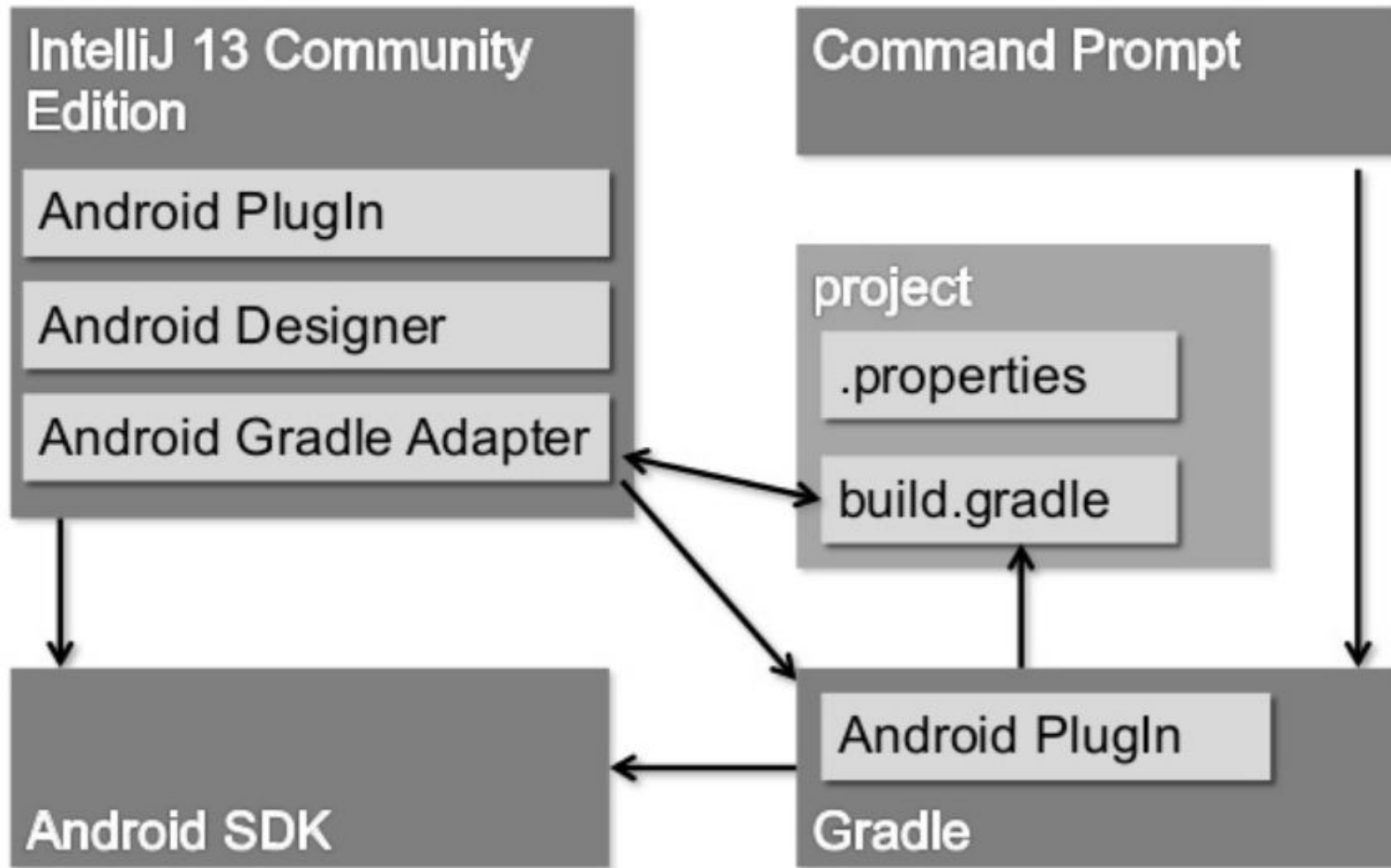
- Android Studio ha importanti vantaggi:
 - Design della UI in parallelo su più form factor
 - Molto più veloce durante l'uso e la compilazione
 - Maggiore integrazione con gli strumenti di debug
 - Integrazione più stretta con Lint



Architetture a confronto



Architetture a confronto



L'emulatore



- **L'AVD Manager** è il punto di avvio per l'**emulatore** di Android
 - Consente di eseguire applicazioni senza necessità di un dispositivo fisico
- Gestisce vere e proprie **macchine virtuali**
 - Diverso hw/os simulati
 - Possibilità di snapshot



Interprete vs. Esecuzione VM



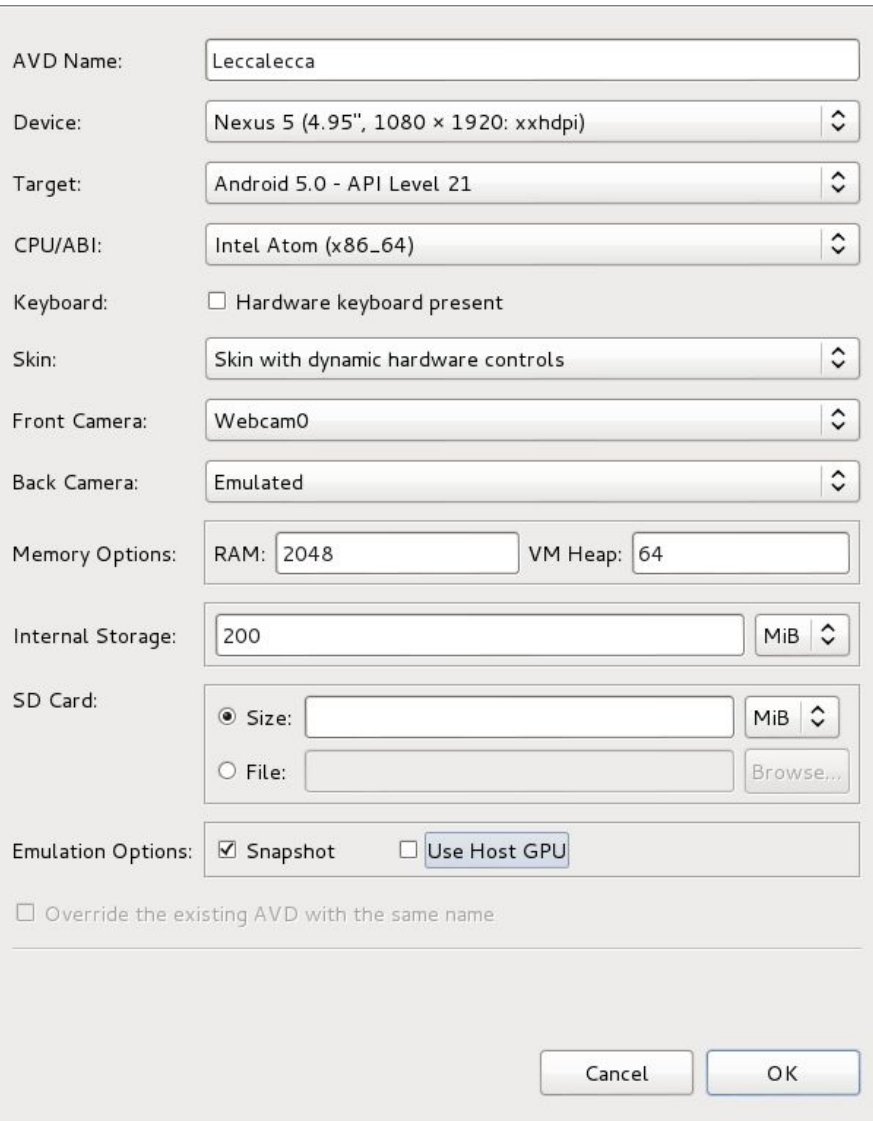
- Problema

- La mamma va al mercato. Compra un PC e uno smartphone. Vuole sviluppare su Android
- Il PC ha un chip Intel, ma lo smartphone ha un ARM
- Come può simulare su PC l'esecuzione su ARM?

- Risoluzione

- Se l'emulatore è configurato con un'immagine ARM, viene eseguita l'app in modo **interpretato** (lento)
- Se l'emulatore è configurato con un'immagine x86 Atom, viene eseguita l'app in modo **nativo** (veloce)
 - Dentro una macchina virtuale a livello hardware – su Linux richiede il modulo **KVM** installato nel kernel

L'emulatore



- Selezionare l'AVD Manager
 - Clic sull'icona!
- Selezionare **Create...**
- Definire le caratteristiche della VM desiderata
 - In particolare: il Target (espresso come API Level)
- Selezionare **OK**

L'emulatore



Skin: 1080x1920

Density: 480

☐ Scale display to real size

Screen Size (in): 5.0

Monitor dpi: 112 ?

Scale: default

☐ Wipe user data

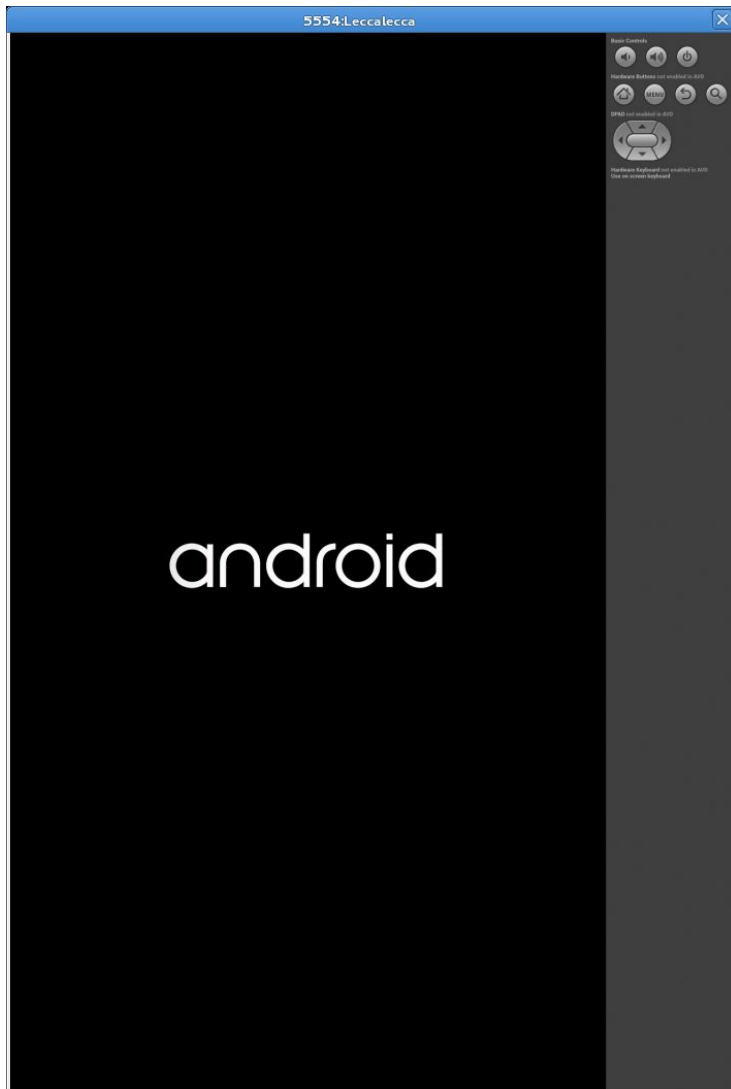
☒ Launch from snapshot

☒ Save to snapshot

Cancel Launch

- Il dispositivo virtuale appena creato si aggiunge alla lista dei dispositivi dell'AVD
- Con Start... si apre una ulteriore finestra di configurazione
 - Si noti l'uso degli snapshot
- Con Launch, si avvia finalmente l'emulatore

Test dell'ambiente

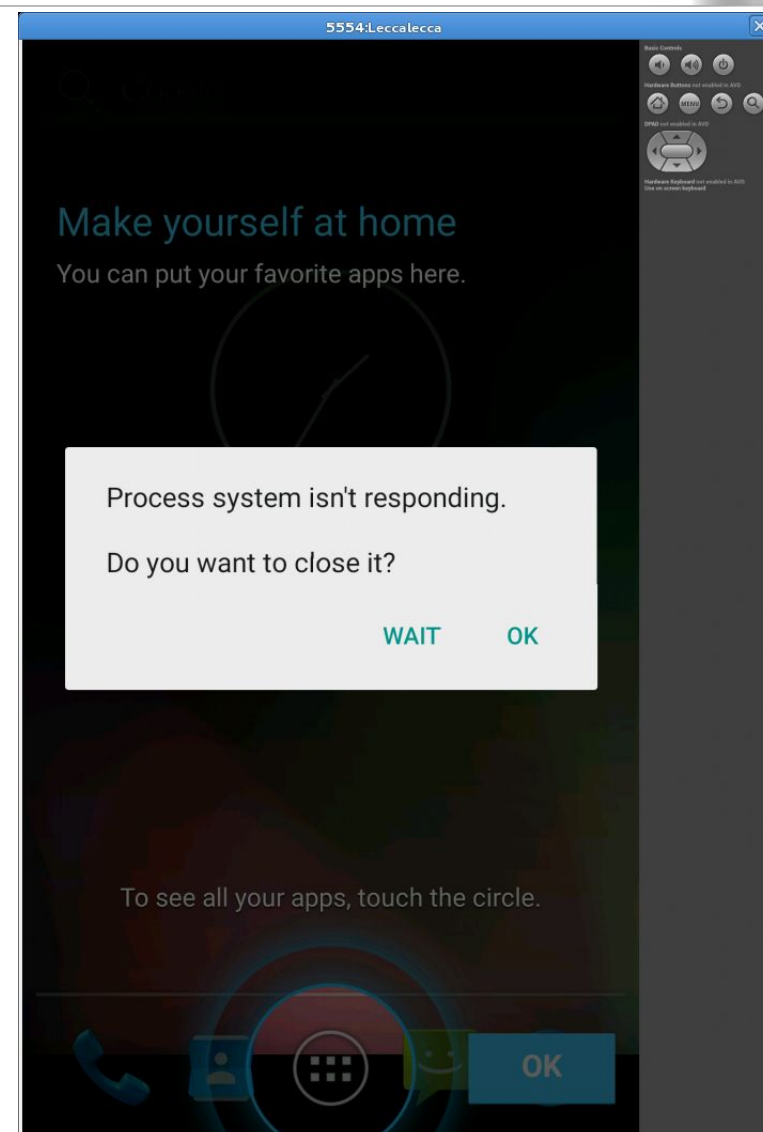


- Pazienza, pazienza...
 - Se usare ARM ABI, l'emulatore sta interpretando il codice ARM istruzione per istruzione!
 - Il primo avvio è un'operazione lenta anche sui telefoni veri
 - Per fortuna, si può abilitare lo snapshot
 - Restore immediato le volte successive

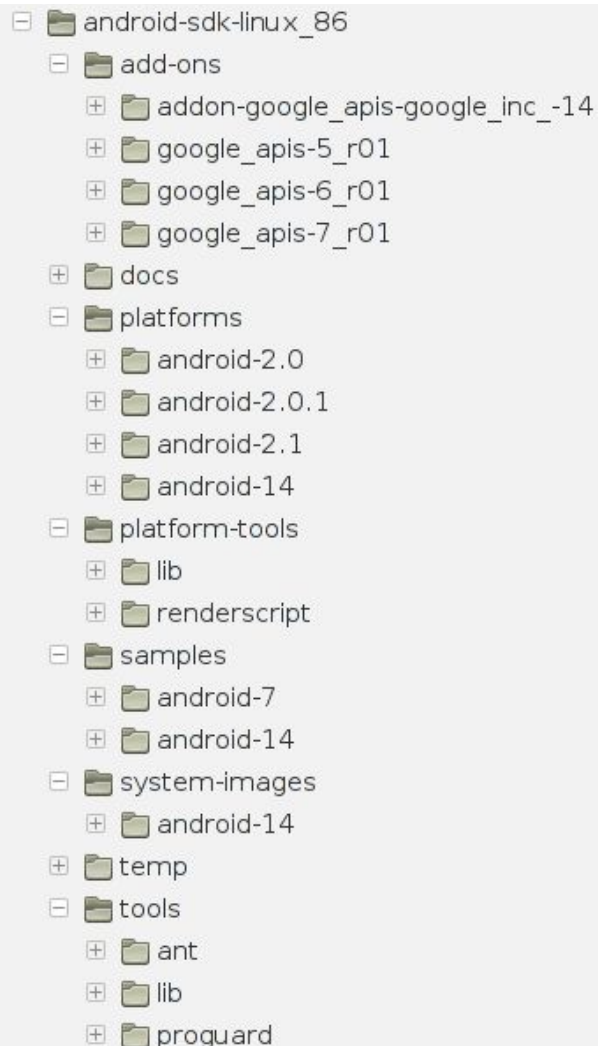
Test dell'ambiente



- **Congratulazioni!**
- Parte il “wizard di primo avvio” di Android
- Se siamo arrivati fin qui... pronti per programmare!
 - Sempre se avete *molta* pazienza...



Esplorare l'SDK



- Potete anche esaminare il contenuto dell'SDK
 - Tool di base
 - Tutti i componenti aggiuntivi installati tramite l'SDK Manager
 - I docs includono copia della documentazione di riferimento disponibile online
 - Stile “Javadocs”, con qualche aggiustamento



Gabriele Pappalardo

Software Developer @ JetBrains |
M.Sc. in Computer Science (Software
Curriculum) | GDG Pisa Community's
Manager

ORGANIZER



Se Android Studio non vi funziona,
non è colpa mia, è colpa **sua**!